HealthMate

Two-Person Work Allocation

Version 1.0 | 6 November 2026 | Kotlin + Jetpack Compose + Room

Repository: github.com/navod718/healthmate. Use only this Navod718 repository. Person labels are placeholders; no collaborators are added by this plan.

1. Ownership overview

Workstream	Person 1: UI / Presentation	Person 2: Data / Logic
Branch	feature/compose-ui	feature/room-data
Foundation	Compose theme, navigation, reusable components	Room configuration, entity, DAO, repository
Dashboard	Summary cards, date display, empty states	Today totals and reactive data
Records	List, date picker, edit entry point, delete dialog	Date filtering, reads and deletion
Add / Edit	Form, input handling, validation messages	Validation rules, insert and update
State	Bind screens to ViewModels and render UI state	ViewModels, StateFlow and loading/error states
Tests	Compose UI and navigation tests	Validation, repository and Room tests
Report evidence	Wireframes, screenshots and UI/UX decisions	Architecture, database, CRUD and package decisions

Shared: Agree interfaces and wireframes, integrate features, run end-to-end tests, review the report, build the release APK and verify submission links. Effort should be reviewed after the first working feature; these are complementary responsibilities, not guaranteed equal hours.

2. Coursework context

CW1.docx labels the assessment individual coursework. This allocation does not establish permission for joint assessed implementation. Apply the lecturer's rules on collaboration and acknowledge assistance as required. The demonstration must contain the submitting student's own voice, and actual AI use must be declared.

3. Person 1 - UI and presentation

Branch: feature/compose-ui

• Create the Material 3 colour scheme, typography, reusable cards, fields and buttons.

• Build Dashboard, Records and Add/Edit screens, plus delete confirmation.

• Implement navigation and date selection; provide previews with sample screen state while data work is in progress.

• Display field errors, loading indicators, empty states and save failures. Preserve form input when a save fails.

• Connect UI callbacks to the agreed ViewModel actions. Keep database queries out of composables.

• Check readable text, touch targets, smaller screens and increased font sizes.

• Prepare wireframes, real screenshots, navigation evidence and UI/UX report notes.

Completion criteria: Every required screen is navigable; forms display agreed validation errors; edit/delete actions work through ViewModels; dashboard and list render real data; UI tests and manual checks have actual results.

4. Person 2 - data and application logic

Branch: feature/room-data

• Define health_records as a Room entity with id, date, steps, calories and water.

• Create RoomDatabase and DAO methods for insert, read, update, delete, date filtering and daily totals.

• Implement the repository with suspend writes and Flow reads; keep database work off the main thread.

• Implement ViewModels and StateFlow for records, selected date, dashboard and form state.

• Validate required non-negative whole numbers, at least one positive health value and no future date.

• Provide explicit demo-data loading that avoids unintended duplicate seeding; surface recoverable errors.

• Prepare the architecture diagram, database diagram, selected code explanations and meaningful logic/database test evidence.

Completion criteria: CRUD persists across restart; date filtering is correct; multiple daily entries aggregate correctly; data changes update UI state; tests document actual outcomes.

5. Shared contract - agree before coding

Contract	Agreed behavior
Record fields	id: Long; date: YYYY-MM-DD string; steps, calories, water: Int. Water is in ml.
Daily entries	Allow multiple records per day. Today totals sum matching records; an empty day shows zero.
Repository operations	Observe records with optional date filter; observe totals for date; fetch by id; insert; update; delete; load demo records.
UI state	Explicit loading, data, empty, field-error and operation-error states.
Screen actions	Select date, add, edit, save, request/cancel/confirm delete, retry and load demo data.
Date behavior	Use the device local date for today; refresh totals when the app resumes after a date change.
Validation ownership	Person 2 defines reusable rules; Person 1 displays results and supplies suitable input controls.

6. Integration and Git workflow

• Both: agree wireframes, contracts and the base Android package before implementation.

• Person 2: prepare the minimal compilable Kotlin/Compose project foundation on feature/room-data. Review and merge this foundation into main first.

• Person 1: update feature/compose-ui from main, then implement screens using sample state. Person 2 continues the data layer and ViewModels.

• Both: commit meaningful increments. Avoid editing each other's owned files; coordinate build configuration changes.

• Open pull requests into main. Review and integrate the data contracts before wiring the UI to real records.

• After each merge, update the other branch from main and resolve conflicts before continuing.

• Together: run add, edit, delete, filter, totals, persistence and empty/demo-data checks on the integrated app.

7. Final handoff checklist

Person 1 supplies UI evidence and report notes. Person 2 supplies database and logic evidence. Together verify the release APK installs, the public repository contains complete source, the report uses the required format, and the Google Drive APK/video links work. Record real test results and preserve genuine Git history. This document is a work plan, not evidence of completed implementation.